Network Programming - 18
Network Library 확장

afewhee@gmail.com

- 암호화 / 복호화
- ODBC → DB 연결
- ODBC → DB 연결
- MySQL 연결

- 주요 내용
  - 네트워크 라이브러리 이후 확장 계층
  - 패킷 계층
  - 암호화 / 복호화 계층
  - DB 접근 계층
  - ODBC / OLEDB 기반 DB 라이브러리
  - ODBC Driver 기반 확장 가능성

1.1 Network Library 확장 방향


```
Application
    ↓
Network Library
    ↓
Packet Layer
    ↓
Encryption Layer
    ↓
Socket Layer

Server Logic
    ↓
DB Library
    ↓
ODBC / OLEDB
    ↓
Database
```

```
DB Library
├ ILcDB
├ CLcDB
├ CLcOdbc
└ CLcOledb
```

```
Encryption Layer
├ ILcCrypt
├ CLcCrypt
├ CLcXorCrypt
└ 외부 암호화 라이브러리 후보
```

```
Packet Flow
├ PacketEncode
├ Encrypt
├ Send
├ Recv
├ Decrypt
└ PacketDecode
```

- 설계 기준
  - 공통 사용 함수 분리
  - 인터페이스 기반 사용 규칙
  - 구현 클래스 교체 가능 구조
  - 응용 계층 코드 단순화
  - DB 종류와 접근 방식 은닉
  - 암호화 알고리즘 교체 가능 구조
  - 네트워크 패킷 처리 흐름 유지

추상화
 ILcDB
 ILcCrypt

공통 기반
 CLcDB
 CLcCrypt

구체화
 CLcOdbc / CLcOledb
 CLcXorCrypt / 외부 라이브러리

- ODBC
  - Open Database Connectivity
  - SQL 기반 DB 접근
  - DB 접근 표준 인터페이스
  - ODBC Driver 기반 연결
  - DSN / SQL Server / Access / Excel 연결
  - C 스타일 API 중심

- OLE DB
  - Object Linking and Embedding Database
  - COM 기반 데이터 접근
  - Provider 기반 연결
  - ADO _ConnectionPtr, _RecordsetPtr 사용
  - SQL Server / Excel / 기타 데이터 소스 연결

1.5 ODBC 주요 함수

- SQLAllocHandle
  → Environment / Connection / Statement 핸들 생성
- SQLSetEnvAttr → ODBC 버전 속성 설정
- SQLConnect → DSN / SQL Server 연결
- SQLExecDirect → SQL 문 실행
- SQLNumResultCols → 결과 컬럼 수 획득
- SQLBindCol → 결과 컬럼과 버퍼 연결
- SQLFetch → 결과 행 이동
- SQLCloseCursor → SQL 결과 커서 종료
- SQLDisconnect → DB 연결 종료
- SQLFreeHandle → ODBC 핸들 해제

1.6 OLEDB / ADO 주요 함수

- CoInitialize → COM 초기화
- CoUninitialize → COM 해제
- _ConnectionPtr::CreateInstance
  → Connection 객체 생성
- ConnectionString → 연결 문자열 설정
- Connection::Open → DB 연결
- _RecordsetPtr::CreateInstance
  → Recordset 객체 생성
- Recordset::Open → SQL 실행과 결과 집합 생성
- Recordset::GetFields → 결과 필드 목록 획득
- Fields::GetCount → 결과 컬럼 수 획득
- Fields->Item[i]->GetValue → 컬럼 값 획득
- Recordset::MoveNext → 다음 행 이동
- Recordset::Close → 결과 집합 종료
- Connection::Close → DB 연결 종료

- 설계 방향
  - DB 접근 코드의 라이브러리화
  - ODBC / OLEDB 구현 차이 은닉
  - SQL 실행 흐름 통일
  - 결과 바인딩 방식 통일
  - 응용 계층은 ILcDB 기준 접근
  - DB 연결 방식은 Factory에서 선택

2.1 DB Library 설계 : Application 흐름


```
Application
     ↓
ILcDB
     ↓
CLcDB
     ↓
CLcOdbc / CLcOledb
     ↓
ODBC API / OLEDB ADO
     ↓
Database
```

- 주요 내용
  - DB 객체 생성 규칙
  - DB 연결 규칙
  - SQL 실행 규칙
  - 결과 바인딩 규칙
  - ODBC / OLEDB 공통 사용 인터페이스

2.2 ILcDB 인터페이스 구현 내용


```
interface ILcDB
{
    virtual int Create(void* p1=nullptr,
                       void* p2=nullptr,
                       void* p3=nullptr,
                       void* p4=nullptr)=0;
    virtual void Destroy()=0;
    virtual int  Query(char* sCmd, void* pData)=0;
    virtual int  Connect(void* pDataType,
                       void* p1=nullptr,
                       void* p2=nullptr,
                       void* p3=nullptr)=0;
    virtual void Close()=0;
    virtual int  SqlBind(char* sSQL,
                       char*** sOutputBufc,
                       int** nDataBufc,
                       int nBufSize)=0;
    virtual int  SqlExec()=0;
    virtual int  SqlClose()=0;
};
```

2.3 ILcDB 함수 구성

- 인터페이스 구성
  - Create → DB 객체 초기화
  - Destroy → 전체 자원 정리
  - Connect → DB 연결
  - Close → 연결 종료
  - SqlBind → SQL 실행 준비 + 결과 버퍼 연결
  - SqlExec → 결과 행 단위 처리
  - SqlClose → SQL 커서와 결과 버퍼 정리
  - Query → 구현별 추가 명령
- 구조적 특징
  - 네트워크 ILcNet과 동일한 설계 방향
  - 사용 규칙 추상화
  - 구현 클래스 교체 가능성
  - 응용 코드와 DB API 분리

- 처리 과정
  - 문자열 명령 기반 DB 구현 선택
  - "ODBC" → CLcOdbc
  - "OLEDB" → CLcOledb
  - 생성 후 Create
  - 연결까지 필요한 경우 CreateAndConnect

2.4 DB Factory


```
int LnDB_Create(char* sCmd, ILcDB** pData) {
    *pData = nullptr;
    if(0==_stricmp("ODBC", sCmd)) {
        CLcOdbc * pDB=nullptr;
        pDB = new CLcOdbc;
        if(FAILED(pDB->Create())) {
            delete pDB;
            return -1;
        }
        *pData = pDB;
        return 1;
    }
    else if(0==_stricmp("OLDEDB", sCmd)) {
        CLcOledb * pDB=nullptr;
        pDB = new CLcOledb;
        if(FAILED(pDB->Create())) {
            delete pDB;
            return -1;
        }
        *pData = pDB;
        return 1;
    }
    return -1;
}
```

- 처리 과정
  - DB 구현 선택
  - DB 객체 생성
  - 연결 인자 전달
  - OLEDB Host 설정 가능
  - 실패 시 Close + delete
  - 성공 시 ILcDB 포인터 반환

2.5 CreateAndConnect


```
int LnDB_CreateAndConnect(char* sCmd, ILcDB** pData,
                          void* p1, void* p2, void* p3, void* p4, void* p5) {
    *pData = nullptr;
    if(0==_stricmp("ODBC", sCmd)) {
        CLcOdbc * pDB=nullptr;
        pDB = new CLcOdbc;
        if(FAILED(pDB->Create(p1, p2, p3, p4))) {
            pDB->Close();
            delete pDB;
            return -1;
        }
        *pData = pDB;
        return 1;
    } else if(0==_stricmp("OLEDB", sCmd)) {
        CLcOledb * pDB=nullptr;
        pDB = new CLcOledb;
        if(p5)
            pDB->Query("Set Host", p5);
        if(FAILED(pDB->Create(p1, p2, p3, p4))) {
            pDB->Close();
            delete pDB;
            return -1;
        }
        *pData = pDB;
        return 1;
    }
    return -1;
}
```


- 구현 구조
  - ILcDB 구현 기반
  - DB Type 관리
  - Host / DSN / UID / Password 보관
  - 실제 DB 접근은 파생 클래스에서 구현
  - 네트워크의 CLcNet과 같은 위치

```
class CLcDB : public ILcDB {
public:
    enum EDBaseTyte {
        DB_NONE=0,
        DB_MDB, DB_DSN, DB_SQL, DB_EXL,
        DB_TOT,
    };

protected:
    EDBaseTyte m_eType{};
    char m_sHst[260]{};
    char m_sDsn[260]{};
    char m_sUid[260]{};
    char m_sPwd[260]{};
};
```

- 설계 의미
  - 공통 상태만 보관
  - 실제 DB API 호출 없음
  - ODBC / OLEDB 파생 클래스에서 구체화

```
int CLcDB::Create(void* p1, void* p2, void* p3, void* p4)
{
    return -1;
}
void CLcDB::Destroy()
{
}
int CLcDB::Connect(void* pDataType, void* p1, void* p2, void* p3)
{
    return -1;
}
int CLcDB::SqlBind(char* sQuery,
                   char*** sDataBufc,
                   int** nDataBufc,
                   int nBufSize)
{
    return -1;
}
```

4.1 ODBC 구현: CLcOdbc 구조

- 구현 구조
  - CLcDB 상속
  - ODBC 환경 핸들
  - ODBC 연결 핸들
  - ODBC Statement 핸들
  - 결과 필드 개수
  - 결과 문자열 버퍼
  - 결과 길이 버퍼

```
class CLcOdbc : public CLcDB
{
protected:
    HANDLE m_hEnv;    // SQLHENV
    HANDLE m_hDbc;    // SQLHDBC
    HANDLE m_hStm;    // SQLHSTMT

    SHORT m_nField;
    char** m_pDataBuf;
    int*   m_nDataBuf;
};
```

4.2 ODBC 연결 종류

- 연결 방식
- DB_DSN → DSN 연결
- DB_SQL → SQL Server 연결
- DB_MDB → Access MDB 연결
- DB_EXL → Excel 파일 연결

```
protected:
    int ConnectDsn(char* sDsn);
    int ConnectSql(char* sDsn,
                   char* sUID,
                   char* sPWD);

    int ConnectMdb(char* sFile);
    int ConnectExcel(char* sFile);
```

4.4 CLcOdbc Create

- 처리 과정
  - ODBC 환경 핸들 할당
  - ODBC 버전 속성 설정
  - DB 연결 핸들 할당
  - 연결 인자 존재 시 Connect 호출

4.5 CLcOdbc Create 구현


```
int CLcOdbc::Create(void* p1, void* p2, void* p3, void* p4) {
    SQLRETURN hr;
    hr = SQLAllocHandle(SQL_HANDLE_ENV,
                        SQL_NULL_HANDLE,
                        &m_hEnv);
    if (FAILED(hr))
        return -1;
    hr = SQLSetEnvAttr(m_hEnv,
                       SQL_ATTR_ODBC_VERSION,
                       (SQLPOINTER)SQL_OV_ODBC3,
                       SQL_IS_INTEGER);
    if (FAILED(hr))
        return -1;
    hr = SQLAllocHandle(SQL_HANDLE_DBC,
                        m_hEnv,
                        &m_hDbc);
    if (FAILED(hr))
        return -1;
    if(p1)
        hr = Connect(p1, p2, p3, p4);
    if (FAILED(hr))
        return -1;
    return 1;
}
```


- 처리 과정
  - DB 타입 문자열 확인
  - mdb, dsn, sql, excel
  - 내부 타입 DB_MDB, DB_DSN, DB_SQL, DB_EXL
  - 타입별 연결 함수 호출

4.7 CLcOdbc Connect 구현


```
int CLcOdbc::Connect(void* pDataType, void* p1, void* p2, void* p3) {
    SQLRETURN hr;
    char* sType = (char*)pDataType;
    hr = SQLAllocHandle(SQL_HANDLE_DBC, m_hEnv, &m_hDbc);
    if (FAILED(hr))
        return -1;
    if(0==_stricmp("mdb", sType))
        m_eType = DB_MDB;
    else if(0==_stricmp("dsn", sType))
        m_eType = DB_DSN;
    else if(0==_stricmp("sql", sType))
        m_eType = DB_SQL;
    else if(0==_stricmp("excel", sType))
        m_eType = DB_EXL;

    if(DB_NONE == m_eType)
        return -1;
    else if(DB_MDB == m_eType)
        return ConnectMdb((char*)p1);
    else if(DB_DSN == m_eType)
        return ConnectDsn((char*)p1);
    else if(DB_SQL == m_eType)
        return ConnectSql((char*)p1, (char*)p2, (char*)p3);
    else if(DB_EXL == m_eType)
        return ConnectExcel((char*)p1);
    return -1;
}
```

- 처리 과정
  - DSN 문자열 준비
  - 사용자 ID / Password 준비
  - SQLConnect
  - Statement 핸들 할당

4.9 ODBC DSN / SQL 연결 구현


```
int CLcOdbc::ConnectDsn(char* sDsn) {
    SQLCHAR InCon[260]={0};
    SQLRETURN hr;
    sprintf((char*)InCon, sDsn);
    hr = SQLConnect(m_hDbc,
                    InCon, SQL_NTS,
                    (SQLCHAR*)"", SQL_NTS,
                    (SQLCHAR*)"", SQL_NTS);
    if (FAILED(hr))
        return -1;
    hr = SQLAllocHandle(SQL_HANDLE_STMT,
                        m_hDbc,
                        &m_hStm);

    if (FAILED(hr))
        return -1;
    return 1;
}
```

```
int CLcOdbc::ConnectSql(char* sDsn,
                        char* sUID,
                        char* sPWD)
{
    SQLCHAR InCon[260]={0};
    SQLCHAR sqUID[128]={0};
    SQLCHAR sqPWD[128]={0};
    SQLRETURN hr;
    sprintf((char*)InCon, sDsn);
    sprintf((char*)sqUID, sUID);
    sprintf((char*)sqPWD, sPWD);
    hr = SQLConnect(m_hDbc,
                    InCon, SQL_NTS,
                    sqUID, SQL_NTS,
                    sqPWD, SQL_NTS);
    if (FAILED(hr))
        return -1;
    hr = SQLAllocHandle(SQL_HANDLE_STMT,
                        m_hDbc,
                        &m_hStm);
    if (FAILED(hr))
        return -1;
    return 1;
}
```

- 처리 과정
  - SQL 문자열 준비
  - SQLExecDirect
  - 결과 컬럼 수 확인
  - 컬럼 개수만큼 버퍼 할당
  - 모든 결과를 문자열로 바인딩
  - 출력 버퍼 포인터 반환

4.11 ODBC SqlBind 구현


```
int CLcOdbc::SqlBind(char* sQuery, char*** sDataBufc,
                     int** nDataBufc, int nBufSize) {
    SQLRETURN hr;
    int i=0;
    SQLCHAR sSQL[1024];
    sprintf((char*)sSQL, sQuery);

    hr = SQLExecDirect(m_hStm, sSQL, SQL_NTS);
    SQLNumResultCols(m_hStm, &m_nField);

    m_pDataBuf = new char*[m_nField]{};
    m_nDataBuf = new int  [m_nField]{};
    for(i=0; i<m_nField; ++i) {
        m_pDataBuf[i] = new char[nBufSize]{};
    }

    for(i=0; i<m_nField; ++i) {
        char* sDataBuf = m_pDataBuf[i];
        hr = SQLBindCol(m_hStm, i+1, SQL_C_CHAR,
                        (SQLPOINTER)(sDataBuf), nBufSize,
                        (SQLINTEGER*)(&m_nDataBuf[i]));
        if(FAILED(hr))
            break;
    }
    *sDataBufc = m_pDataBuf;
    *nDataBufc = m_nDataBuf;
    return m_nField;
}
```

- 처리 과정
  - SqlExec
  - SQLFetch
  - 행 단위 결과 이동
  - SQL_NO_DATA면 종료

```
int CLcOdbc::SqlExec()
{
    SQLRETURN hr;
    hr = SQLFetch(m_hStm);
    if(SQL_NO_DATA == hr)
        return -1;
    return 1;
}
```

4.13 ODBC SqlClose

- 정리 과정
  - 커서 닫기
  - 컬럼 버퍼 해제
  - 길이 버퍼 해제
  - 필드 수 0 초기화

```
int CLcOdbc::SqlClose() {
    SQLRETURN hr=-1;
    hr = SQLCloseCursor(m_hStm);
    if(m_pDataBuf) {
        for(int i=0; i<m_nField; ++i)
            delete [] m_pDataBuf[i];
        delete [] m_pDataBuf;
        m_pDataBuf = nullptr;
    }
    if(m_nDataBuf) {
        delete [] m_nDataBuf;
        m_nDataBuf = nullptr;
    }
    m_nField = 0;
    return hr;
}
```

5.1 OLEDB 구현: CLcOledb 구조

- 구현 구조
  - CLcDB 상속
  - ADO Connection
  - ADO Recordset
  - 결과 필드 개수
  - 결과 문자열 버퍼
  - 결과 길이 버퍼

```
class CLcOledb : public CLcDB
{
protected:
   _ConnectionPtr m_pCon;
   _RecordsetPtr  m_pRsc;
   SHORT m_nField;
   char** m_pDataBuf;
   int*   m_nDataBuf;
};
```

5.2 OLEDB 연결 종류: 구현

- 연결 방식
  - SQL Server
  - Excel
  - ODBC DSN
  - OLEDB DSN
  - 독립 DSN

```
protected:
   int ConnectSql(char* sDataBase,
                  char* sUID,
                  char* sPWD);
   int ConnectExcel(char* sFile);

   int ConnectDsnOdbc(char* sDSN,
                      char* sUID,
                      char* sPWD);

   int ConnectDsnOledb(char* sDSN,
                       char* sUID,
                       char* sPWD);

   int ConnectDsnInd(char* sDSN,
                     char* sUID,
                     char* sPWD);
```

- 처리 과정
  - COM 초기화
  - 연결 인자 존재 시 Connect
  - Recordset Close
  - Connection Close
  - COM 해제

5.4 CLcOledb Create 구현


```
int CLcOledb::Create(void* p1,
                     void* p2,
                     void* p3,
                     void* p4)
{
    int hr=-1;

    if(FAILED(::CoInitialize(nullptr)))
        return -1;

    if(p1)
        hr = Connect(p1, p2, p3, p4);

    if (FAILED(hr))
        return -1;

    return 1;
}
```

5.5 CLcOledb Destroy 구현


```
void CLcOledb::Destroy()
{
    if(m_pRsc)
    {
        m_pRsc->Close();
        m_pRsc = nullptr;
    }

    if(m_pCon)
    {
        m_pCon->Close();
        m_pCon = nullptr;
    }

    ::CoUninitialize();
}
```

- 처리 과정
  - "Set Host" 명령
  - Host 문자열 저장
  - SQL Server 연결 문자열 생성 시 사용

```
int CLcOledb::Query(char* sCmd, void* pData)
{
    if(0==_stricmp("Set Host", sCmd))
    {
        char* sHst = (char*)pData;

        strcpy(m_sHst, sHst);
        return 0;
    }

    return -1;
}
```

```
if(p5)
    pDB->Query("Set Host", p5);
```

- 처리 과정
  - 타입 문자열 확인
  - mdb, dsn, sql, excel
  - 현재 구현 중심: SQL / Excel
  - SQL은 ConnectSql
  - Excel은 ConnectExcel

5.8 CLcOledb Connect 구현


```
int CLcOledb::Connect(void* pDataType,
                      void* p1, void* p2, void* p3) {
    char* sType = (char*)pDataType;

    if(0==_stricmp("mdb", sType))
        m_eType = DB_MDB;
    else if(0==_stricmp("dsn", sType))
        m_eType = DB_DSN;
    else if(0==_stricmp("sql", sType))
        m_eType = DB_SQL;
    else if(0==_stricmp("excel", sType))
        m_eType = DB_EXL;

    if(DB_NONE == m_eType)
        return -1;
    else if(DB_SQL == m_eType)
        return ConnectSql((char*)p1, (char*)p2, (char*)p3);
    else if(DB_EXL == m_eType)
        return ConnectExcel((char*)p1);

    return -1;
}
```

- 처리 과정
  - 연결 문자열 생성
  - SQLOLEDB Provider
  - Data Source
  - Initial Catalog
  - User Id / Password
  - Connection 객체 생성
  - Open

5.10 OLEDB SQL 연결 구현


```
int CLcOledb::ConnectSql(char* sDataBase, char* sUID, char* sPWD)
{
    char sCon[512]={0};
    strcpy(m_sDsn, sDataBase);
    strcpy(m_sUid, sUID);
    strcpy(m_sPwd, sPWD);

    if(strlen(m_sUid) && strlen(m_sPwd))
        sprintf(sCon,
            "provider=SQLOLEDB; "
            "Data Source=%s; "
            "Initial Catalog=%s; "
            "User Id=%s;Password=%s; ",
            m_sHst, m_sDsn, m_sUid, m_sPwd);
    else
        sprintf(sCon,
            "provider=SQLOLEDB; "
            "Data Source=%s; "
            "Initial Catalog=%s; "
            "User Id=%s;Password=; ",
            m_sHst, m_sDsn, m_sUid);

    ComValidTest(m_pCon.CreateInstance(__uuidof(Connection)));
    m_pCon->ConnectionString = sCon;
    m_pCon->ConnectionTimeout = 30;
    m_pCon->Open("","","",adConnectUnspecified);

    return 1;}
```

- 처리 과정
  - Microsoft.Jet.OLEDB.4.0 Provider
  - Excel 파일 경로
  - Extended Properties
  - Connection 객체 생성
  - Open

5.12 OLEDB Excel 연결 구현


```
int CLcOledb::ConnectExcel(char* sFile)
{
    char sCon[512]={0};

    sprintf(sCon,
        "provider=Microsoft.Jet.OLEDB.4.0; "
        "Data Source= %s ; "
        "Extended Properties=\"Excel 8.0; "
        "HDR=No; ReadOnly=False\" ",
        sFile);

    ComValidTest(m_pCon.CreateInstance(__uuidof(Connection)));

    m_pCon->ConnectionString = sCon;
    m_pCon->ConnectionTimeout = 10;
    m_pCon->Open("","","",adConnectUnspecified);

    return 1;
}
```

- 처리 과정
  - SQL 문자열 준비
  - Recordset 생성
  - Recordset Open
  - Fields 접근
  - 필드 개수 확인
  - 결과 버퍼 할당
  - 결과 버퍼 포인터 반환

5.14 OLEDB SqlBind 구현


```
int CLcOledb::SqlBind(char* sQuery, char*** sDataBufc, int** nDataBufc, int nBufSize)
{
    HRESULT hr = -1;
    char sSQL[1024]={0};
    sprintf(sSQL, sQuery);
    hr = ComValidTest(m_pRsc.CreateInstance(__uuidof(Recordset)));
    hr = m_pRsc->Open(sSQL,
                      m_pCon.GetInterfacePtr(),
                      adOpenForwardOnly,
                      adLockReadOnly,
                      adCmdText);
    FieldsPtr pFldLoop = nullptr;
    pFldLoop = m_pRsc->GetFields();
    m_nField = pFldLoop->GetCount();
    m_pDataBuf = new char*[m_nField];
    m_nDataBuf = new int  [m_nField];

    for(LONG i=0; i<m_nField; ++i)
    {
        m_pDataBuf[i] = new char[nBufSize];
        memset(m_pDataBuf[i], 0, nBufSize);
    }

    *sDataBufc = m_pDataBuf;
    *nDataBufc = m_nDataBuf;
    return m_nField;
}
```

- 처리 과정
  - EndOfFile 확인
  - 필드별 값 획득
  - 문자열 버퍼 변환
  - 다음 레코드 이동
  - 버퍼 해제
  - Recordset Close

5.16 OLEDB SqlExec 구현


```
int CLcOledb::SqlExec()
{
    if(m_pRsc->EndOfFile)
        return -1;

    for(LONG i=0; i<m_nField; ++i)
    {
        _variant_t var;
        char* sDataBuf = m_pDataBuf[i];

        var = m_pRsc->Fields->Item[i]->GetValue();

        // variant 값 → 문자열 버퍼
    }

    m_pRsc->MoveNext();

    return 1;
}
```

5.17 OLEDB SqlClose 구현


```
int CLcOledb::SqlClose() {
    if(m_pDataBuf) {
        for(int i=0; i<m_nField; ++i)
            delete [] m_pDataBuf[i];
        delete [] m_pDataBuf;
        m_pDataBuf = nullptr;
    }
    if(m_nDataBuf) {
        delete [] m_nDataBuf;
        m_nDataBuf = nullptr;
    }
    m_nField = 0;
    if(m_pRsc) {
        m_pRsc->Close();
        m_pRsc = nullptr;
    }
    return 1;
}
```

- 처리 과정
  - DB 종류 설정
  - 파일 또는 DSN 설정
  - SQL 문자열 설정
  - LnDB_CreateAndConnect
  - SqlBind
  - SqlExec 반복
  - SqlClose
  - Close

6.1 DB 사용: ODBC 사용 예


```
ILcDB* g_pDataBase=nullptr;

int main()
{
    char sDBtype[32];
    char sFile[MAX_PATH];
    char sSQL[512];

    strcpy(sDBtype, "excel");
    strcpy(sFile, "database/user.xls");
    strcpy(sSQL, "select name,skill,hp from [Sheet1$]");

    if(FAILED(LnDB_CreateAndConnect("ODBC", &g_pDataBase, sDBtype, sFile)))
        return -1;

    int nBufSize = 1024;
    char** sDataBuf = nullptr;
    int* nDataBuf = nullptr;
    int iColumn = 0;

    iColumn = g_pDataBase->SqlBind(sSQL, &sDataBuf, &nDataBuf, nBufSize);
}
```

- 처리 과정
  - 컬럼 수 확인
  - SqlExec 성공 동안 행 처리
  - 컬럼별 문자열 출력
  - SqlClose
  - Close
  - 객체 삭제

6.3 ODBC 결과 처리 구현


```
if(SUCCEEDED(iColumn)) {
    printf("이름              경험치                    체력\n");
    printf("-----------------------------------------\n");

    while(1) {
        if(FAILED(g_pDataBase->SqlExec()))
            break;
        for(int i=0; i<iColumn; ++i) {
            printf("%12s(%2d)\t", sDataBuf[i], nDataBuf[i]);
        }
        printf("\n");
    }
    g_pDataBase->SqlClose();
    printf("-----------------------------------------\n\n");
}

g_pDataBase->Close();
delete g_pDataBase;
```

- 처리 과정
  - OLEDB 선택
  - Excel 연결
  - SQL 실행
  - 결과 바인딩
  - 행 단위 결과 처리
  - Destroy 포함

6.5 OLEDB 사용 구현 예


```
ILcDB* g_pDataBase=nullptr;

int main()
{
    char sDBtype[32];
    char sFile[MAX_PATH];
    char sSQL[512];

    strcpy(sDBtype, "excel");
    strcpy(sFile, "database/user.xls");
    strcpy(sSQL, "Select * from [Sheet1$]");

    if(FAILED(LnDB_CreateAndConnect("OLEDB", &g_pDataBase, sDBtype, sFile)))
        return -1;

    int nBufSize = 1024;
    char** sDataBuf = nullptr;
    int* nDataBuf = nullptr;
    int iColumn = 0;

    iColumn = g_pDataBase->SqlBind(sSQL, &sDataBuf, &nDataBuf, nBufSize);

    // SqlExec 반복
    // SqlClose
    // Close
    // Destroy
}
```


6.6 DB Library 요약

- ILcDB → DB 사용 규칙
- CLcDB → 공통 상태
- CLcOdbc → ODBC 기반 구현
- CLcOledb → OLEDB / ADO 기반 구현
- LnDB_CreateAndConnect → 구현 선택 + 연결
- SqlBind → SQL 실행 준비 + 결과 버퍼 연결
- SqlExec → 행 단위 결과 처리
- SqlClose → SQL 결과 정리
-

- 설계 방향
  - 패킷 데이터 보호
  - 평문 메시지 노출 방지
  - PacketEncode 이후 암호화
  - recv 이후 복호화
  - PacketDecode 이전 평문 복원
  - 암호화 알고리즘 교체 가능 구조

송신
Application Data
↓
PacketEncode
↓
Encrypt
↓
Send

수신
Recv
↓
Decrypt
↓
PacketDecode
↓
Application Data

- 처리 과정

기존 흐름

Send
↓
PacketEncode
↓
rbSnd
↓
send

확장 흐름

Send
↓
PacketEncode
↓
Encrypt
↓
rbSnd
↓
send

수신 확장

recv
↓
rbRcv
↓
Decrypt
↓
PacketDecode
↓
Recv

- 설계 기준
  - 패킷 전체 암호화
  - Payload만 암호화
  - Header 평문 유지 여부 선택
  - 패킷 크기 확인 방식과 충돌 주의

7.4 Header와 Payload 암호화 구현 방식 비교

- 방식 1 : [Header] [Encrypted Payload]
  - 장점: 패킷 크기 확인 쉬움, 수신 버퍼 처리 단순
  - 단점: Header 정보 노출
- 방식 2: [Encrypted Header + Payload]
  - 장점: 정보 노출 감소
  - 단점: 패킷 크기 확인 어려움, 수신 버퍼 처리 복잡
- 기본 방향
  - Header 평문 + Payload 암호화
  - 실전 확장 → 별도 보안 프로토콜 또는 검증 구조 필요

- 설계 의미
  - 암호화 사용 규칙
  - 알고리즘 구현 은닉
  - XOR / 외부 라이브러리 교체 가능
  - 네트워크 라이브러리와 분리된 계층

7.6 ILcCrypt 인터페이스 구현 구조


```
#ifndef interface
#define interface struct
#endif

interface ILcCrypt
{
    virtual int  Create(void* pKey=nullptr,
                        void* pOption=nullptr)=0;

    virtual void Destroy()=0;

    virtual int Encrypt(BYTE* pOut,
                        int* pOutSize,
                        const BYTE* pIn,
                        int iInSize)=0;

    virtual int Decrypt(BYTE* pOut,
                        int* pOutSize,
                        const BYTE* pIn,
                        int iInSize)=0;
};
```

- 설계 의미
  - 공통 Key 저장
  - Key 크기 관리
  - 알고리즘별 파생 클래스 기반

7.8 CLcCrypt 공통 기반 구현 구조


```
class CLcCrypt : public ILcCrypt
{
protected:
    BYTE m_sKey[256]{};
    int  m_iKeySize{};

public:
    CLcCrypt() { }
    virtual int Create(void* pKey=nullptr, void* pOption=nullptr)
    {
        if(pKey) {
            m_iKeySize = (int)strlen((char*)pKey);
            memcpy(m_sKey, pKey, m_iKeySize);
        }
        return 0;
    }
    virtual void Destroy()
    {
        memset(m_sKey, 0, sizeof(m_sKey));
        m_iKeySize = 0;
    }
};
```

- 특징
  - Encrypt와 Decrypt 동일
  - 실제 보안 목적 부적합
  - 패킷 처리 위치 설명에 적합

7.10 XOR 암호화 구조 – 구현


```
class CLcXorCrypt : public CLcCrypt
{
public:
    virtual int Encrypt(BYTE* pOut,
                        int* pOutSize,
                        const BYTE* pIn,
                        int iInSize)
    {
        if(nullptr == pOut || nullptr == pIn || m_iKeySize <= 0)
            return -1;
        for(int i=0; i<iInSize; ++i)
            pOut[i] = pIn[i] ^ m_sKey[i % m_iKeySize];
        if(pOutSize)
            *pOutSize = iInSize;
        return iInSize;
    }

    virtual int Decrypt(BYTE* pOut, int* pOutSize, const BYTE* pIn, int iInSize)
    {
        return Encrypt(pOut, pOutSize, pIn, iInSize);
    }
};
```

7.11 PacketEncode와 Encrypt 결합


```
int CLcNet::SendEncrypted(const char* pSrc,
                          int iSnd,
                          DWORD dMsg)
{
    BYTE sPacket[PCK_BUF_MAX_MSG] = {0};
    BYTE sCrypt [PCK_BUF_MAX_MSG] = {0};

    int iPacketSize =
           LcNet_PacketEncode(sPacket,
                      pSrc, iSnd, dMsg);

    int iCryptSize = 0;
    m_pCrypt->Encrypt(sCrypt,
                      &iCryptSize,
                      sPacket,
                      iPacketSize);

    m_rbSnd.PushBack(sCrypt, iCryptSize);

    return 0;
}
```

구현 위치

Send

↓

PacketEncode

↓

Encrypt

↓

rbSnd.PushBack

7.12 Decrypt와 PacketDecode 결합


```
int CLcNet::RecvEncrypted(char* pDst,
                          int* iRcv,
                          DWORD* dMsg)
{
    BYTE sCrypt [PCK_BUF_MAX_MSG] = {0};
    BYTE sPacket[PCK_BUF_MAX_MSG] = {0};

    WORD iCryptSize = 0;
    int  iPacketSize = 0;

    if(FAILED(m_rbRcv.PopFront(sCrypt,
&iCryptSize)))
        return -1;

    m_pCrypt->Decrypt(sPacket,
                      &iPacketSize,
                      sCrypt,
                      iCryptSize);

    *iRcv =
        LcNet_PacketDecode((BYTE*)pDst,
            dMsg, sPacket, iPacketSize);

    return 0;
}
```

구현 위치

rbRcv.PopFront

↓

Decrypt

↓

PacketDecode

↓

Application Data

7.13 암호화 적용 단위 – 구현 방식 비교

- Payload 암호화

```
[Length][Kind][Encrypted Message][Tail]
```

- 장점
  - 기존 RingBuffer 구조 유지
  - Length 기반 PopFront 가능
- 단점
  - Kind 값 노출

7.14 암호화 적용 단위 – 구현 방식 비교

- Payload + Kind 암호화

```
[Length][Encrypted Kind + Message][Tail]
```

- 장점
  - 메시지 종류 노출 감소
- 단점
  - Decode 순서 변경 필요

- 전체 패킷 암호화

```
[Encrypted Length + Kind + Message + Tail]
```

- 장점
  - 정보 노출 최소화
- 단점
  - 패킷 크기 판별 구조 변경 필요

7.16 암호화 라이브러리 후보

- 직접 구현
  - XOR
  - 구조 설명용
  - 보안 목적 부적합
- 외부 라이브러리
  - libsodium
  - Crypto++
  - OpenSSL EVP
- ILcCrypt 유지
  - 내부 구현만 외부 라이브러리로 교체
- 응용 계층
  - Encrypt / Decrypt 직접 호출 최소화

7.17 암호화 계층 설계 요약

- ILcCrypt → 암호화 사용 규칙
- CLcCrypt → 공통 Key 상태
- CLcXorCrypt → 구조 설명용 구현
- 외부 암호화 라이브러리 → 실전 구현 후보
- Network Library → ILcCrypt 포인터 보관
- Send: PacketEncode → Encrypt → rbSnd
- Recv: rbRcv → Decrypt → PacketDecode

8.1 통합 구조: Network + DB + Crypt 통합

구현 구조

Application
↓
ILcNet
↓
PacketEncode
↓
ILcCrypt::Encrypt
↓
Socket Send

Socket Recv
↓
ILcCrypt::Decrypt
↓
PacketDecode
↓
Server Logic
↓
ILcDB
↓
ODBC / OLEDB
↓
Database

- 처리 계층
  - Network Layer → 접속 / 송수신
  - Packet Layer → 메시지 형식
  - Crypt Layer → 데이터 보호
  - DB Layer → 계정 / 캐릭터 / 기록 조회

```
Client
    Login Packet 생성
    PacketEncode
    Encrypt
    Send
```

```
Server
    Recv
    Decrypt
    PacketDecode
    Login Request 처리
    ILcDB::SqlBind
    ILcDB::SqlExec
    결과 Packet 생성
    Encrypt
    Send
```

8.3 채팅 로그 저장 예시

- DB 사용 위치
  - 실시간 처리
    - Broadcast
  - 비동기 저장 대상
    - Chat Log
    - Login Log
    - Error Log
    - Game Result

처리 과정

```
Client Chat
     ↓
Encrypt Packet
     ↓
Server Recv
     ↓
Decrypt
     ↓
PacketDecode
     ↓
Broadcast
     ↓
DB Log Insert
```


- 공통 설계 방향
  - 인터페이스 추상화
  - 공통 기반 클래스
  - 구체 구현 클래스
  - Factory 선택
  - 응용 계층 코드 단순화

```
Network Library
    ILcNet
    CLcNet
    Select / Event / IOCP
DB Library
    ILcDB
    CLcDB
    CLcOdbc
    CLcOledb
Crypt Layer
    ILcCrypt
    CLcCrypt
    CLcXorCrypt
    외부 라이브러리 교체 가능
```

9. 정리

- 네트워크 → 송수신 추상화
- DB → 데이터 접근 추상화
- 암호화 → 패킷 보호 계층
- 공통 방향 → 인터페이스 기반 라이브러리화

```
Application
    ↓
ILcNet
    ↓
PacketEncode
    ↓
ILcCrypt
    ↓
Socket
```

```
Socket
    ↓
ILcCrypt
    ↓
PacketDecode
    ↓
Server Logic
    ↓
ILcDB
    ↓
CLcOdbc / CLcOledb
    ↓
Database
```